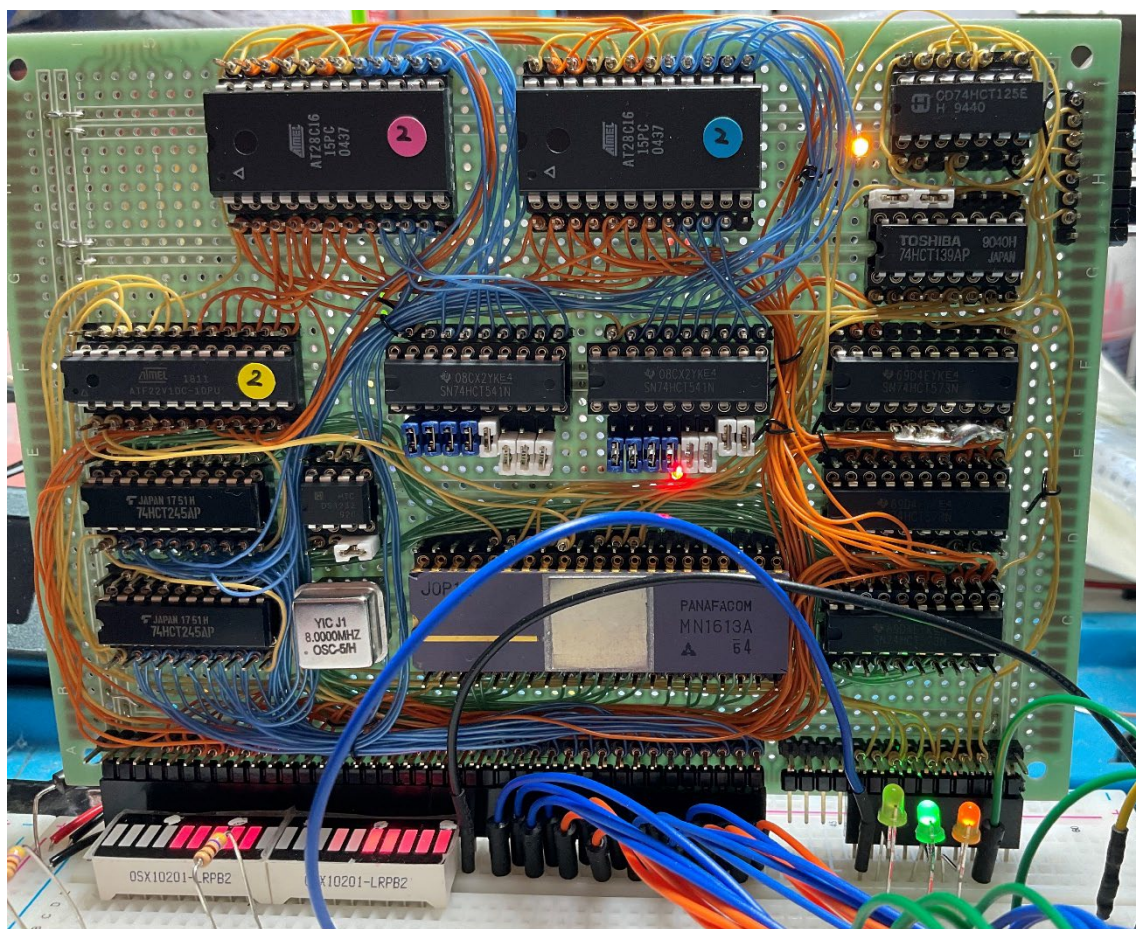


MN1613 マイコンボードの製作

—製作・動作確認編—



Mercury 日本橋

MUTIF Project

内容

はじめに	3
設計段階からの変更点	3
アドレス・データ分離	3
メモリ・I/O 制御信号生成	3
クロック源	4
動作状態確認用 LED	4
アドレスデコーダの搭載	4
リセット時のスタートアドレス（リセットベクタ）生成回路	5
リセット回路	5
ステップ実行回路追加	6
制御信号の取り扱い（プルアップ・プルダウン）	6
ラッチアップ対策	7
動作確認	8
その他	10

表紙の写真：製作した MN1613 基板のテスト中の様子。写真右上の黄色の LED の点灯は現在ステップ実行モードであること、中央の LED の点灯は MN1613 が実行中であることを示している。左下のバーLED はアドレスバスに出ている値をビットパターンで表示している。テストプログラムを実行中であり、アドレス値の各ビットに対応する LED も点灯・消灯を繰り返すため、濃淡が出ているように見えている。

はじめに

本冊子は、パナファコム独自開発の、日本初の 1 チップ 16 ビットマイクロプロセッサである MN1610 の後継である MN1613 を使ったマイコンボードの MN1613 基板部分¹の実際の製作と動作確認までのまとめとなります。MN1610 や MN1613 の特徴、マイコンボードの設計の概要については、前回（設計編）を参照してください。なお、変更後、すなわち動作確認が取れた回路図については、本冊子の付録を、そして、回路図を含む設計情報に打ち手は、GitHub でも公開・更新予定（下記）ですので、そちらも参照いただければと思います。

<https://github.com/ja1umi?tab=repositories>

設計段階からの変更点

アドレス・データ分離

MN1613 は 16 ビット長のバスをアドレスとデータで共用しています。こうした設計は MN1613 に特有のものではなく、例えば Intel の 8088 でも同様です。そこで、データとバスの分離についても比較的豊富な作例がある 8088 マイコンボードのそれを参考にすることができます。

MN1613 はバスにアドレス値を出力している間 ADSD を HIGH にするので、それを利用してアドレスをラッチします。当初、8×D-FF の 74LS574 を 2 つ使うこととしていましたが、transparent latch である 74HCT573²を使うようにしています。 $\overline{OE}$ を方向制御信号として使って 74HCT234 を 2 つでバスバッファを入れるところは変えていません。

メモリ・I/O 制御信号生成

$\overline{DTSD}$ 、IOP、WRT を 2-to-4 ラインデコーダの 74HCT139 でデコードして、メモリ読み書き用の信号と I/O 読み書き用の信号を生成しています。 $\overline{DTSD}$ を使うのは、アドレス値ではなく、データ値を扱うことを保証するためです。また、I/O は後述するようにリセット後のスタートアドレスを求める際にも必要になるので、たとえユーザープログラムで I/O 空

¹ マイコンボード自体は、CPU 基板（ここでは MN1613 基板と、汎用メモリボード C.E.L.E.S.T.E; CPU-independent Essential Learning Equipment to be Subjected to Tests and Experiments for old CPUs（CPU に依存しない古い CPU のテストと実験のための必須学習機器、/se'leste/と発音）から構成されますが、本冊子で扱うのは MN1613 基板だけです。

² LS ではなく HCT にしているのは単に好みです。

間をアクセスすることがなくともデコードしておく必要があります。

クロック源

当初、安定動作のため 2 MHz のクリスタルオシレータを使う予定としていました。実際試してみると 8 MHz までは大丈夫でしたが、10 MHz では動作が不安定になりました。ワイヤラッピング配線ではこのあたりが限界なのかもしれません。

動作状態確認用 LED

途中まで面発光 LED アレイを使っていましたが、回路追加の場所確保のためと、この面発光 LED アレイが思ったより暗くて視認性がいまいちだったので、秋月電子で手に入る 1.8mm のアキシアルリード型 LED に変更しました。2.2k Ω の電流制限抵抗を介して MN1613 の $\overline{CSRQ/RUN}$ と +5V と繋いでいます。LED のアノード側を +5V にしているので、MN1613 が RUN 状態になると点灯します (active LOW)。

アドレスデコーダの搭載

この種の目的に汎用されてきた GAL (SPLD) の入手性がここにきてようやく改善されて以前のように入手できるようになったため³、アドレスデコーダは FPGA 基板ではなく、

GAL22V10			
nWR	1	24	VCC
IOP	2	23	NC
A15	3	22	nCERAM
A14	4	21	nCEROM
A13	5	20	nCESHM
A12	6	19	nMBINT
A11	7	18	A1
A10	8	17	A2
A9	9	16	A3
A8	10	15	A4
A7	11	14	A5
GND	12	13	A6

ATF22V10C を用いることにし MN1613 基板と一緒に載せるように変更しました。22V10 は I/O ピン数が多めなので、16 ビットアドレスでもほぼフルデコードできます。

22V10 や 16V8 のロジック定義には WinCUPL を用いることが一般的ですが、前に GI 社の CP-1600 を使ったブレッドボードマイコンを製作した時から Galette⁴に乗り換えました。GALasm を Rust で書き換えたもので、Linux 上でも、そして CLI で使えるので重宝します。

左の図は Galette がソースリストのピン定義部分から自走生成したピン配置図です。ピン定義は 1 番ピンから順に書いていくので実際のピン配置と違ってくるので、こうした機能は地味ですが便利です。

Galette の紹介も兼ねて、現時点でのアドレスデコーダの定義 (pld ファイル) を示します。これを Galette で処理すると、WinCUPL と同様、jed ファイルが生成されるので、これを

³ 一時期は ATF22V10 などリードタイムが約 1 年とかでした。

⁴ <https://github.com/simon-frankau/galette>

ユニバーサルプログラマで書き込むことになります。

```
GAL22V10
adecode1613      ; Address decoder

nWR      IOP      A15      A14      A13      A12      A11      A10      A9      A8      A7      GND
A6        A5        A4        A3        A2        A1      nMBINT nCESHM nCEROM nCERAM nMBINTA VCC

; Shared RAM : 0x?0000 - 0x?0FFF (?=0, 1, 2)
/nCESHM = /A15 * /A14 * /A13 * /A12 * /IOP

; ROM : 0x?F800 - 0x?FFFF (?=0, 1, 2)
/nCEROM = A15 * A14 * A13 * A12 * A11 * /IOP

; RAM : 0x01000 - 0x0F7ff, 0x11000 - 0x1F7FF, 0x21000 - 0x2F7FF, 0x31000 - 0x3F7FF
/nCERAM = nCESHM * nCEROM * /IOP

; Mail Box : 0x?0042 (?=0, 1, 2)
; MBINT is read at the rising edge of nWR and is cleared when nMBINTA is asserted (active low).
/nMBINT.R = /nCESHM * /A11 * /A10 * /A9 * /A8 * /A7 * A6 * /A5 * /A4 * /A3 * /A2 * A1
AR = /nMBINTA
```

DESCRIPTION

These equations are used for implementing the address decoder for MN1613 CPU Board

Galette を使って書き直したアドレスデコーダの定義。文法は GAMasm と（ほぼ）変わらないが、CUPL と異なり、式に括弧は使えないので、全て手動で展開しなければならない。

リセット時のスタートアドレス（リセットベクタ）生成回路

MN1613 はリセット後に I/O 空間のアドレス 0 を読み出して、その値+3 のアドレスのメモリ内容が示すアドレスから実行を開始します。そこで、I/O 読み出しがあったら任意のアドレス値をバスに出力するようにしました(I/O 読み書きの信号をデコードする必要があるのは、まさにこのためです)。具体的には、バスバッファ 74HCT541 の出力と $\overline{OE}$ を、それぞれデータバスと 74HCT139 のデコード出力 (I/O 読み出し) と接続、入力 は 10k Ω の抵抗でプルダウンした上で、ジャンパーピンを介して +5V のレールと接続できるようにしてあります。ジャンパーピンを外せば LOW、装着すれば HIGH になるので、2 個の 541 で合計 16 ビット分のアドレス値を設定できます。I/O 読み出しが出たら、ジャンパーピンで設定した値がバスに出るというわけです。

この結線では I/O 空間の読み出しはデコードしていますが、I/O 空間のアドレスはデコードしていないので、I/O 空間のアドレス 0 でなくとも 541 が enable になってしまいます。従って、事実上 I/O 入力命令はプログラムで使えなくなってしまうのですが、目をつぶることにしました。

リセット回路

初期設計通り、watchdog timer 機能付きのリセット IC である DS1232 を使うことには変

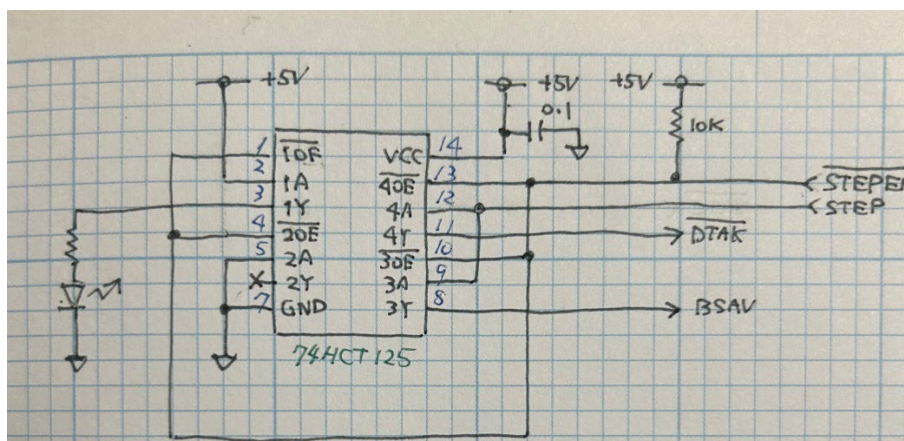
更はありませんが、watchdog timer のリセット用の信号は ADSD ではなく、SYNC⁵を使用しました。これは、後述するステップ実行機能の追加に伴う変更です。

ステップ実行回路追加

MN1613 基板の動作確認中にブレッドボードに組んで使ってみて使っていたのですが、思っていた以上に便利だったので追加したものです（下図）。

設計編にも書きましたが、MN1613 は命令を実行する際、 $\overline{BSRQ}$ と BSAV、 $\overline{DTSD}$ と $\overline{DTAK}$ でハンドシェイクを行います（BSAV は HIGH、 $\overline{DTAK}$ は LOW に固定しても動作します）。これを利用して、3 ステートバッファの 74HCT125 の出力は BSAV と $\overline{DTAK}$ に接続します。74HCT125 の $\overline{OE}$ （下図でいう $\overline{STEPEN}$ ）が HIGH の時は何も変わりませんが、LOW の時は入力（下図でいう STEP）によって BSAV と $\overline{DTAK}$ が状態が決まります。入力が HIGH の時は BSAV も HIGH なのでバスの使用が許可されますが、 $\overline{DTAK}$ が HIGH のため、先に進みません。逆に入力が LOW の時は、 $\overline{DTAK}$ が LOW なので現在の命令は完了できますが、BSAV が HIGH なので、次のバスサイクルの時にバスが使用できず、先に進めません。こうして、ステップ実行ができるわけです。この説明だけでは分かり辛いと思うので、設計編や MN1613 のデータシートのタイミングチャートも参照してみてください。

なお、ステップ実行回路の追加にあたって、余っているバッファを使って、ステップ実行の状態では LED（アキシャルリード型）が点灯するようにしてみました。



制御信号の取り扱い（プルアップ・プルダウン）

実のところ、MN1613 基板は「勝手に HALT する病」「ラッチアップ」に悩まされ続けました。前者についていえば、無限ループのテストプログラムを実行させると、上記の動作状態確認用の LED が一瞬点灯してすぐに消灯します。ステップ実行させてみると、リセット

⁵ SYNC には MN1613 の X2 ピンに入力されたクロック信号の 1/3 の周波数の信号が出力されます。

後、数命令は実行されるものの、その後勝手に HALT していることがわかりました。クロック源のクリスタルオシレータを異なる周波数のものと交換して試してみると、HALT するまでの命令数が変わります。もちろん、MN1613 の $\overline{HLT}$ ピンはプルアップされています(つまり、GND に落ちてはいませんし、フローティングにもなっていません)。

更なる試行で、HALT した状態で $\overline{STRT}$ を HIGH から LOW に落とすと実行が再開されることに気づきました (LOW にしたままでは実行は再開されません)。そこで、 $\overline{CSRQ/RUN}$ と $\overline{STRT}$ を NOT ゲートを挟んで繋いでみると、HALT することはなくなりました。 $\overline{CSRQ/RUN}$ が LOW になる (つまり HALT する) と $\overline{STRT}$ が LOW になるので、HALT したそばから実行が再開されるというわけです。これで勝手に HALT することはなくなりましたが、あくまで「対症療法」で、このままではソフトウェアで H 命令 (HALT 命令) が使えません (実際に使うケースは少なそうですが)。

この怪現象の原因は、実のところ MN1613 で追加された $EA1/\overline{CSTP}$ ピンのプルアップ忘れでした。このピンは ADSD の後に拡張アドレス (アドレスの 17 ビット目、EA1) が出力されるのですが、の出力ピンである一方、 $\overline{DTSD}$ の後は入力モードになります ($\overline{CSTP}$)⁶。手に入ったデータシートでは「停止情報」とあるだけで詳細は不明ですが、入力モードになっている時に本ピンが LOW になる (あるいは、それに加えてその他の条件が揃うと) MN1613 が HALT するのでしょう。 $EA1/\overline{CSTP}$ が入力モードになることを全く考慮しておらず(汗)、アドレス出力とだけ思い込んで 74HCT573 の入力と繋いでいました。このため、 $EA1/\overline{CSTP}$ が入力モードになった時、本ピンはフローティングの状態になり、たまたま LOW になって条件が揃うと HALT したということなのでしょう。対処方法はあっけないほど単純で、 $EA1/\overline{CSTP}$ をプルアップすることでした。本件に関しては、MN1613 について色々実験されていた @electrelic 氏から旧 Twitter (現 X) で指摘を頂いたことが解決に繋がりました。ここで感謝の意を表します。

上述した $\overline{STRT}$ ですが、HALT からの実行再開には HIGH→LOW の遷移が必要なだけで実のところ起動時にプルアップあるいはプルダウンいずれであっても動きます。ただ、何かの際に GND に落とせば強制的に実行再開できるということで、プルアップに変更しました (当初はプルダウン)

他の制御信号のプルアップ・プルダウンについては変更ありません。

ラッチアップ対策

上述した「勝手に HALT する病」と並んで対策に苦慮したのがこれです。最初はステップ動作の回路をブレッドボード側に追加してトラブルシューティングしていたのですが、電源を投入すると高頻度でステップ動作しない状態になるのに気づきました。観察してみると、この状態になった時は動作状態確認用の LED が点灯しません。さらに調べてみると、

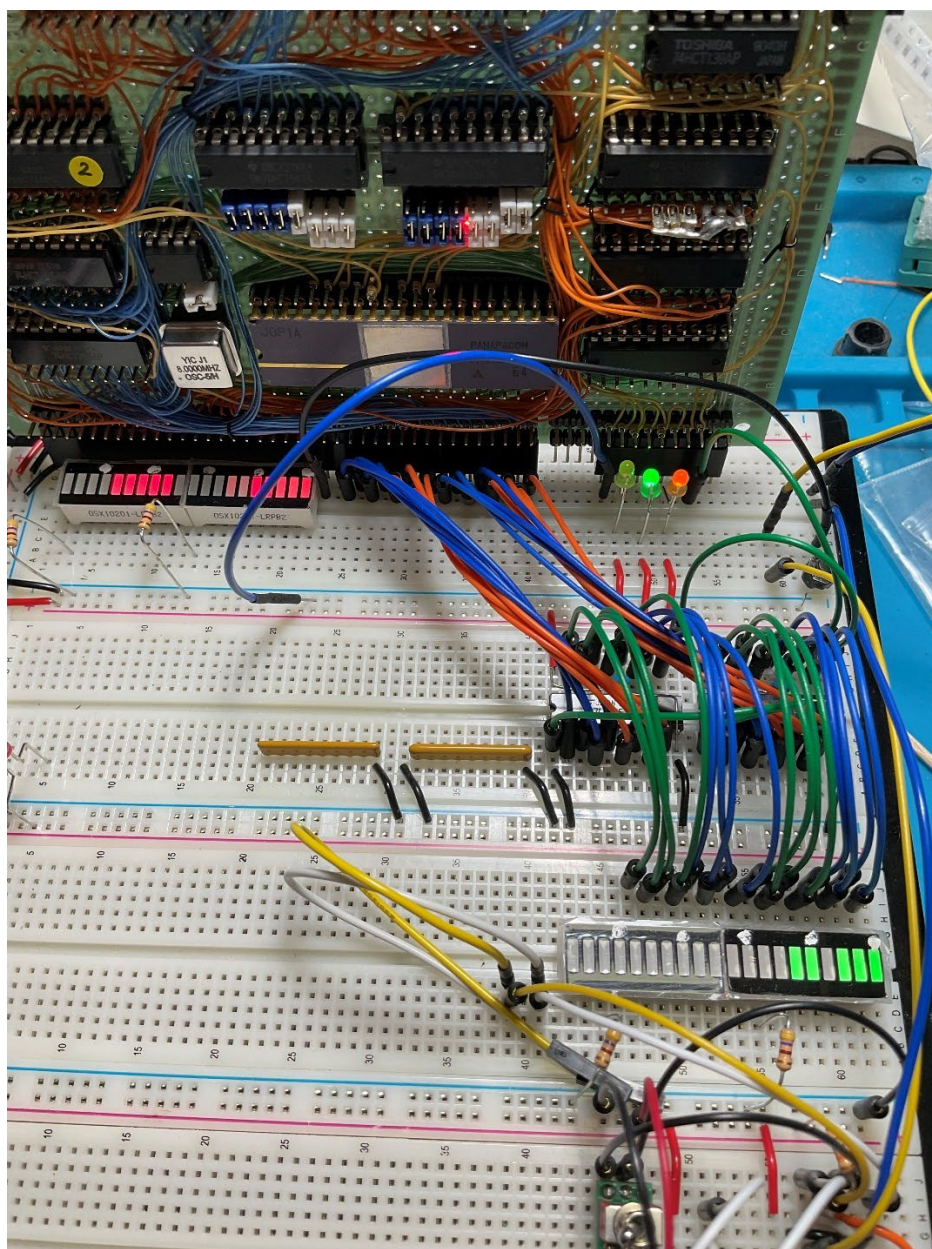
⁶ちなみに、 $EA0/FSYC$ ピンは出力モードのみです。

動作しない状態では動作した状態と比較して、かなり多くの電流が流れていることに気づきました。絵に描いたようなラッチアップです。動作状態確認用の LED が点灯しないことから、①MN1613 が HALT しているか、②クロックが入力されていないか、③あるいは電源 (+5V) 電圧の立ち上がりよりも先にクロック信号が入るため、④クロック信号にのる外来ノイズ、あるいはオーバーシュート・アンダーシュートが関与、といった原因が考えられました。既に上述の「勝手に HALT」への対策は済ませていましたし、クロックオシレータも正常です、電源電圧の立ち上がりについてはオシロスコープでの確認はしていませんが、電源の種類を変えてみても状況は変わりません。ということで、クロック信号のノイズ、オーバーシュート・アンダーシュート対策を試みることにしました。

MN1613 のクロック入力 (X2) ピンの所を小容量 (47pF) の積層セラミックコンデンサで GND に落とす対策は無効でしたが、クロック入力がフローティングになるのを防ぐという意味での同ピンの 330Ωでのプルアップはある程度有効、そしてショットキーバリアダイオード (BAT43) を同ピンと +5V の間に入れる対策が奏功しました。なお、基板の電源と電源の間にフェライトビーズを入れて試しましたが、効果は認められませんでした。本件に関しては、常日頃お世話になっている @kaokun 氏の助言が大いに役立ちました。この場を借りて御礼申し上げます。

動作確認

MN1613 基板は汎用メモリボード C.E.L.E.S.T.E に挿して使うことを前提として設計されていますが、その C.E.L.E.S.T.E は MN1613 基板と同時並行で製作している関係で、まだ完成していません。そこで MN1613 基板の動作確認は、同基板をブレッドボードに立てて、最低限必要な回路をブレッドボードに組んで行うこととしました。具体的には、アドレスバスにはバーLED、データバスには 8×D-FF の 74HCT574 を 2 個挟んで (上位 8 ビットと下位 8 ビット) その先にバーLED を繋ぎました。74HCT 574 のクロック入力と MN1613 基板の $\overline{WR}$ 出力を接続しました。こうすることで、メモリあるいは I/O に書き込んだ内容がバーLED に表示されることになります。実際に動作確認したときの様子を示します。



動作確認のため MN1613 基板をブレッドボードに立て、1 から 10 までの総和をメモリに書き込むプログラムを走らせている様子。プログラム自体は MN1613 基板上の EEPROM (AT28C16) に書き込んである。写真左上の赤色バーLED はアドレスバスの値、写真右下の緑色バーLED はメモリに書き込まれた値を示す。いずれも点灯している LED に相当するビットが立っていることを意味している。従って、緑色バーLED は、ここでは 0b11011 すなわち 10 進数でいう 55 で、正しい結果となっている。

```

5/      0 :
6/      0 :
8/      0 : =X'42'
9/      0 : =X'42'
10/     0 : =X'1000'

MBOX: EQU X'0042'
RESULT: EQU MBOX
RAMLOC: EQU X'1000'

52/ F848 : ROM_PROG2:
53/ F848 : 0800 MVI R0, 0
54/ F849 : 8042 ST R0, RESULT
55/ F84A : 2748 1000 STD R0, RAMLOC
56/ F84C : 090A MVI R1, 10
57/ F84D : 5809 - A R0, R1
58/ F84E : 4141 SI R1, 1, Z
59/ F84F : CFFE B -
60/ F850 : 8042 ST R0, RESULT ; calculate the sum from 1 to 10 and put the result
61/ F851 : 2748 1000 STD R0, RAMLOC
62/ F853 : CFF5 B ROM_PROG2
63/ F854 :

```

動作確認用の 1 から 10 までの総和を求めるプログラム。MN1613 で追加された命令を使用。

その他

今回はトラブルシューティングの関係で MN1613、EEPROM、そしてクリスタルオシレータを頻回抜き差ししました。これらと基板の接続には丸ピン IC ソケットを使っていますが、あきらかに最初と比べて部品が抜きやすくなっているのを実感しています。抜き差しの衝撃と相俟って接触不良や、ソケットと基板のハンダクラックも心配です。現実問題として、MN1613 基板は動作確認中に MN1613 を指で押さえると動作が不安定になる現象が出はじめ、動作確認はできたものの、その後抜き差しを繰り返すにつれ状況は悪化していき、コミックマーケット 104 の時点で動作しなくなっています（涙）。回路はこれで確定したので、プリント基板化する予定で、今後は抜き差しの頻度は格段に減ると思いますが、ZIF ソケットの採用を含め IC ソケットの選定は重要だと改めて痛感しているところです。

今後の開発予定

「設計編」で予告した、動作確認時点での設計情報の更新分や、その時点までに得られた知見の紹介はできましたが、基板の作り直しをする事態となっています。

今後は汎用メモリボード C.E.L.E.S.T.E の製作も進め、C.E.L.E.S.T.E 用のファームウェアを書き、MN1613 基板と C.E.L.E.S.T.E 合わせてマイコンボードとして動作確認したいと思っています。MN1613 以外の CPU 基板、例えば 80286 基板は作りたいですし、そのテストプログラムを書くためにプロテクトモードプログラミングについても自習を進めていますが、まずは MN1613 基板 + C.E.L.E.S.T.E の動作確認が優先ですね。

表 題	： MN1613 マイコンボードの製作 ―製作・動作確認編―		
サークル名	： Mercury 日本橋		
発行者	Kasumi YOSHINO (JA1UMI)	E-mail:	ykasumi@spamex.com
印刷	プリントパック様	X/Twitter:	@yKasumi
		MissKey.io:	@yKasumi
	https://github.com/ja1umi/	BlueSky:	ykasumi.bsky.social
発行日	： 2024 年 8 月 12 日 コミックマーケット 104 (C104)		

